
All Looking Eye

AI-Powered Anomaly Detection System

PROJECT REPORT

*Violence Detection in CCTV Footage using
Deep Learning and Pose Estimation*

Technologies Used

TensorFlow/Keras • MediaPipe • Flask • MongoDB

December 2025

Contents

1	Introduction	3
1.1	Project Overview	3
1.2	Motivation	3
2	System Architecture	3
2.1	High-Level Architecture	3
2.2	Data Flow Diagram	4
2.3	Technology Stack	4
3	Current Implementation	4
3.1	Web Dashboard	4
3.2	Core Detection Pipeline	6
3.3	Model Architecture	6
3.4	Feature Engineering	6
4	Model Development Journey	7
4.1	Datasets Used	7
4.2	Run 1: Simple Statistics Model	7
4.2.1	Approach	7
4.2.2	Feature Extraction	7
4.2.3	Training	8
4.2.4	Result	8
4.2.5	Analysis	8
4.3	Run 2: Sequential Model (LSTM)	8
4.3.1	Approach	8
4.3.2	Feature Extraction	8
4.3.3	Training	9
4.3.4	Result	9
4.3.5	Analysis	9
4.4	Run 3: Smart Features Model (Current Best)	9
4.4.1	Approach	9
4.4.2	Feature Extraction	9
4.4.3	Training	10
4.4.4	Result	10
4.4.5	Analysis	10
4.5	Run 4: Extended Features (Experimental)	10
4.5.1	Approach	10
4.5.2	Result	11
4.5.3	Analysis	11
4.6	Run 5: Quality-Robust Model (Final)	11
4.6.1	Approach	11
4.6.2	Key Improvements	11
4.6.3	Result	12

5	Results and Evaluation	12
5.1	Performance Metrics	12
5.2	Detection Examples	12
6	Conclusion	12
6.1	Key Achievements	12
6.2	Lessons Learned	13
6.3	Future Work	13
A	Training Configuration	13

1 Introduction

1.1 Project Overview

All Looking Eye is an AI-powered anomaly detection system designed to detect violence and fights in CCTV footage. The system uses deep learning techniques, specifically Long Short-Term Memory (LSTM) neural networks combined with MediaPipe pose estimation, to analyze video feeds in real-time and identify potentially dangerous situations.

The primary goal of this project is to build a robust, production-ready system that can:

- Detect fights and violence in video streams
- Provide real-time alert notifications via email
- Support multiple input sources (uploaded videos, webcam, CCTV streams)
- Maintain detection history with confidence scores
- Operate reliably across different video quality levels

1.2 Motivation

Traditional surveillance systems require constant human monitoring, which is prone to fatigue and human error. By leveraging AI and deep learning, we can automate the detection of anomalous behavior, enabling faster response times and more efficient security monitoring.

2 System Architecture

2.1 High-Level Architecture

The system follows a client-server architecture with the following main components:

- **Web Dashboard (Frontend):** A modern web interface for user interaction
- **Video Input Handler:** Processes video from multiple sources
- **Pose Detector:** Extracts body landmarks using MediaPipe
- **LSTM Anomaly Detector:** Neural network for violence classification
- **Alert Generator:** Sends email notifications with video clips
- **Detection Logger:** Stores detection history in MongoDB

2.2 Data Flow Diagram

Figure 1 illustrates the complete data flow through the system:

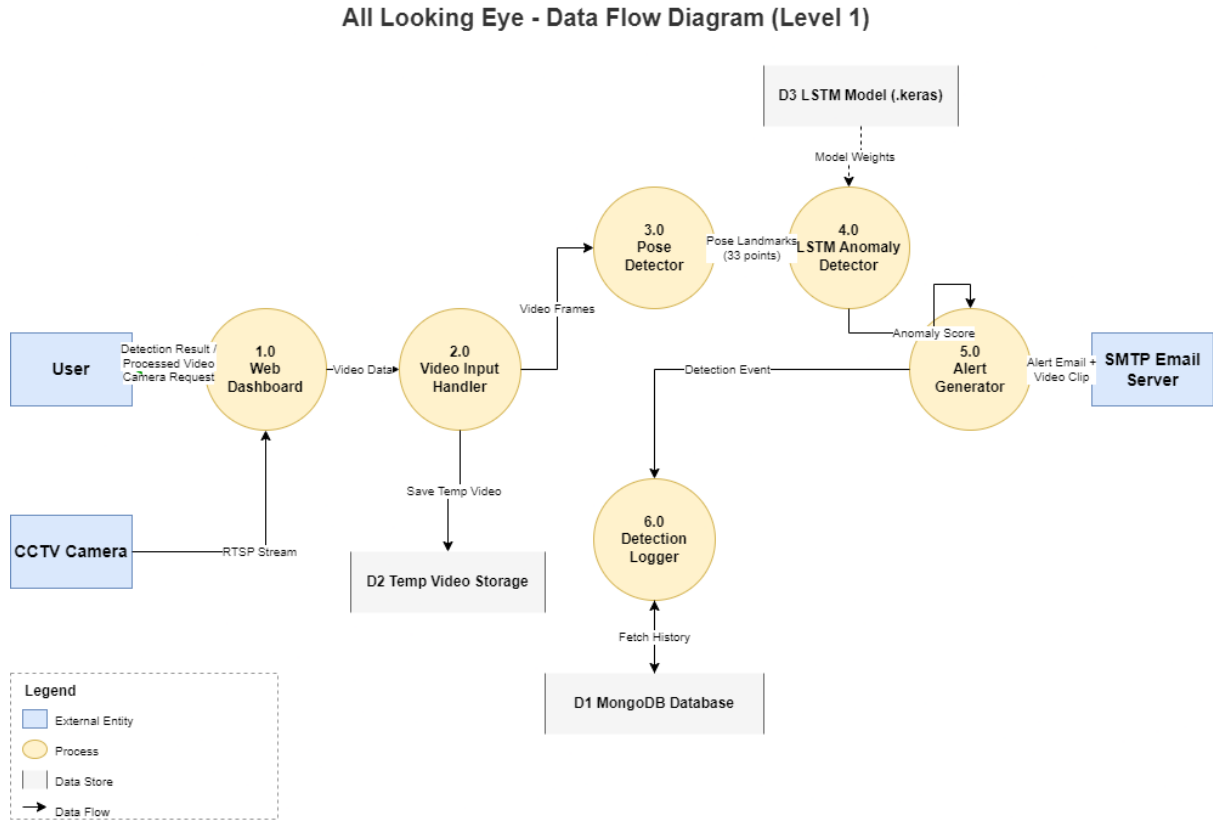


Figure 1: Level 1 Data Flow Diagram showing system components and data flow

2.3 Technology Stack

Component	Technology
Frontend	HTML5, CSS3, JavaScript
Backend	Python, Flask
ML Framework	TensorFlow/Keras
Pose Estimation	MediaPipe
Database	MongoDB
Video Processing	OpenCV
Email Service	SMTP

Table 1: Technology Stack

3 Current Implementation

3.1 Web Dashboard

The web dashboard provides an intuitive interface for users to interact with the anomaly detection system. Key features include:

- **Alert Notifications:** Email configuration for instant alerts when anomalies are detected
- **Multiple Input Sources:**
 - Video file upload (MP4, AVI, MOV formats)
 - Device camera for real-time detection
 - CCTV stream connection via RTSP/HTTP URLs
- **Detection History:** Complete log of all past detections with timestamps, confidence scores, and alert status

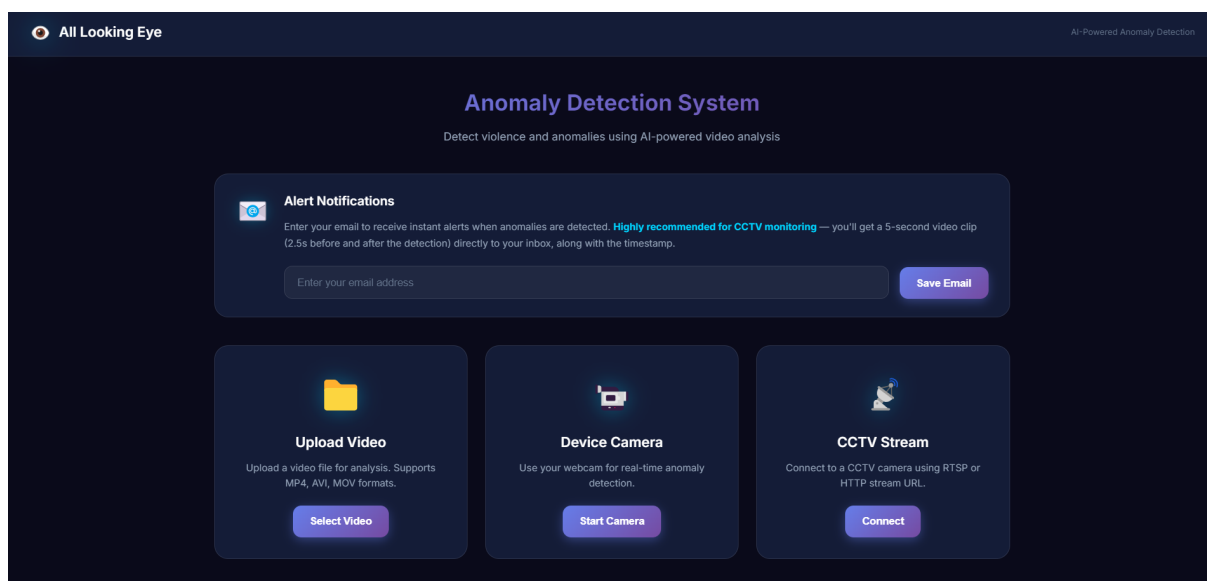


Figure 2: Main Web Dashboard Interface

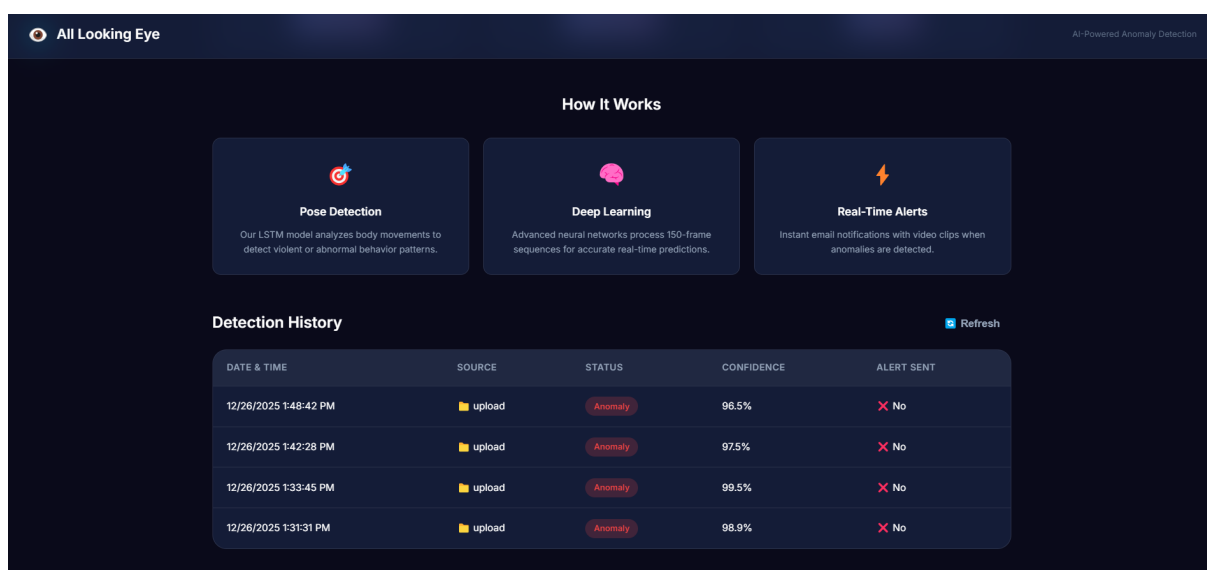


Figure 3: Detection History and How It Works section of the dashboard

3.2 Core Detection Pipeline

The detection pipeline processes video in the following stages:

1. **Frame Extraction:** Video is read frame-by-frame using OpenCV
2. **Pose Estimation:** MediaPipe extracts 33 body landmarks per person
3. **Feature Engineering:** Raw coordinates are transformed into meaningful features
4. **Sequence Processing:** 150 frames are processed as a sequence
5. **Classification:** LSTM model outputs anomaly probability
6. **Alert Generation:** High-confidence detections trigger email alerts

3.3 Model Architecture

The current best-performing model uses a Stacked LSTM architecture:

```
1 model = Sequential()
2 model.add(Input(shape=(150, 264))) # 150 frames, 264 features
3
4 # LSTM Layer 1
5 model.add(LSTM(64, return_sequences=True))
6 model.add(BatchNormalization())
7 model.add(Dropout(0.4))
8
9 # LSTM Layer 2
10 model.add(LSTM(32, return_sequences=False))
11 model.add(BatchNormalization())
12 model.add(Dropout(0.4))
13
14 # Dense Layers
15 model.add(Dense(32, activation='relu'))
16 model.add(Dropout(0.3))
17 model.add(Dense(1, activation='sigmoid'))
```

Listing 1: LSTM Model Architecture

3.4 Feature Engineering

Each frame produces a 264-dimensional feature vector:

- **Raw Pose Features (132):** 33 landmarks $\times$ 4 values (x, y, z, visibility)
- **Velocity Features (132):** Change in position from the previous frame

Advanced preprocessing techniques include:

1. **Normalization:** Centering skeleton on hips and scaling to unit box
2. **Smoothing:** Exponential smoothing to reduce camera jitter
3. **Data Augmentation:** Adding noise to training data for robustness

4 Model Development Journey

This section documents the iterative process of developing the anomaly detection model, including failed approaches and lessons learned.

4.1 Datasets Used

The model was trained on a combination of two publicly available datasets:

- **RWF-2000 Dataset:** 2000 videos (1000 Fight, 1000 NonFight)
- **Real Life Violence Situations Dataset:** Additional violence/non-violence samples

Total training samples: Approximately 4000 videos.

4.2 Run 1: Simple Statistics Model

4.2.1 Approach

The first attempt used a simple statistical approach, hypothesizing that fights could be detected by looking at the "average motion" in a video.

4.2.2 Feature Extraction

- Used MediaPipe to extract 33 body landmarks per frame
- Calculated Mean and Standard Deviation across all frames
- Compressed entire video into a single 264-dimensional vector

```
1 def extract_simple_summary_features(video_path):
2     frame_features_list = []
3     cap = cv2.VideoCapture(video_path)
4
5     while cap.isOpened():
6         success, image = cap.read()
7         if not success: break
8
9         image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
10        results = pose.process(image_rgb)
11
12        if results.pose_landmarks:
13            landmarks = results.pose_landmarks.landmark
14            features = np.array([
15                [lm.x, lm.y, lm.z, lm.visibility]
16                for lm in landmarks
17            ]).flatten()
18            frame_features_list.append(features)
19
20        # Aggregate to single vector
21        video_np = np.array(frame_features_list)
22        mean_features = np.mean(video_np, axis=0)
23        std_features = np.std(video_np, axis=0)
```



```
24
25 return np.concatenate((mean_features, std_features))
```

Listing 2: Simple Feature Extraction

4.2.3 Training

- Trained Random Forest classifier
- Trained XGBoost classifier
- Applied RandomizedSearchCV for hyperparameter tuning

4.2.4 Result

Accuracy: 78%

4.2.5 Analysis

The model failed to distinguish between "fast dancing" and "fighting" because averaging the data destroyed the temporal pattern of movement. The sequential nature of violent actions was completely lost.

Key Lesson: Temporal information is critical for violence detection.

4.3 Run 2: Sequential Model (LSTM)

4.3.1 Approach

The second attempt preserved temporal information by feeding the sequence of movements to an LSTM network.

4.3.2 Feature Extraction

- Extracted raw pose landmarks for every frame (no averaging)
- Padded/trimmed every video to exactly 150 frames
- Output Shape: (150, 132) per video

```
1 def process_and_fix_length(frames_np, max_frames=150):
2     if frames_np is None or frames_np.shape[0] == 0:
3         return np.zeros((max_frames, 132))
4
5     num_frames = frames_np.shape[0]
6
7     if num_frames == max_frames:
8         return frames_np
9     elif num_frames > max_frames:
10        # Sample evenly
11        indices = np.linspace(0, num_frames-1, max_frames, dtype=int)
12        return frames_np[indices]
13    else:
14        # Pad with zeros
15        pad_width = ((0, max_frames - num_frames), (0, 0))
```

```
16 return np.pad(frames_np, pad_width, 'constant')
```

Listing 3: Sequential Feature Extraction

4.3.3 Training

Trained an LSTM neural network using TensorFlow/Keras with the sequential data.

4.3.4 Result

Accuracy: 78%

4.3.5 Analysis

Despite preserving temporal information, the raw coordinate data was too "noisy." The model couldn't identify what a "punch" looked like just from raw pixel positions moving around. Raw coordinates lacked semantic meaning.

Key Lesson: Raw coordinates need to be transformed into meaningful features.

4.4 Run 3: Smart Features Model (Current Best)

4.4.1 Approach

The breakthrough came from calculating meaningful physics-based features, specifically **velocity** – the change in position between frames.

4.4.2 Feature Extraction

- **Raw Pose Features (132):** Position of each landmark
- **Velocity Features (132):** Change in position from previous frame
- Total: 264 features per frame
- Output Shape: (150, 264) per video

```
1 def extract_advanced_features(video_path):
2     frame_features_list = []
3     previous_features = None
4
5     cap = cv2.VideoCapture(video_path)
6     while cap.isOpened():
7         success, image = cap.read()
8         if not success: break
9
10        image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
11        results = pose.process(image_rgb)
12
13        if results.pose_landmarks:
14            landmarks = results.pose_landmarks.landmark
15
16            # Raw features
17            current_features = np.array([
18                [lm.x, lm.y, lm.z, lm.visibility]
```

```
19         for lm in landmarks
20     ]).flatten()
21
22     # Velocity features
23     if previous_features is not None:
24         velocity = current_features - previous_features
25     else:
26         velocity = np.zeros(132)
27
28     # Combine
29     combined = np.concatenate((current_features, velocity))
30     frame_features_list.append(combined)
31     previous_features = current_features
32
33     return np.array(frame_features_list)
```

Listing 4: Advanced Feature Extraction with Velocity

4.4.3 Training

- Stacked LSTM architecture (two LSTM layers)
- Batch Normalization for training stability
- Dropout (0.4) for regularization
- Adam optimizer with learning rate scheduling
- Early stopping to prevent overfitting

4.4.4 Result

Accuracy: 81.5%

4.4.5 Analysis

Velocity provided a clear signal for aggressive movement. The model could now distinguish between:

- Slow, controlled movements (walking, standing)
- Fast, erratic movements (punching, kicking, pushing)

Key Insight: Velocity is the critical feature for violence detection.

4.5 Run 4: Extended Features (Experimental)

4.5.1 Approach

Attempted to add even more semantic features:

- **Angle Features (4):** Joint angles for elbows and knees
- **Distance Features (5):** Distances between key landmarks (wrists, shoulders, nose)
- Total: 273 features per frame

4.5.2 Result

Accuracy: 76% (Decreased!)

4.5.3 Analysis

Adding more features caused overfitting. The model memorized training data specifics rather than learning generalizable patterns. The velocity-only approach remained superior.

Key Lesson: More features better performance. Feature engineering requires careful validation.

4.6 Run 5: Quality-Robust Model (Final)

4.6.1 Approach

The final iteration focused on making the model robust to different video qualities, addressing the domain shift problem between high-quality training data and low-quality CCTV footage.

4.6.2 Key Improvements

```

1 def normalize_pose_landmarks(landmarks):
2     lm_array = np.array([
3         [lm.x, lm.y, lm.z, lm.visibility]
4         for lm in landmarks
5     ])
6
7     # Center on hips
8     center_x = (lm_array[23, 0] + lm_array[24, 0]) / 2
9     center_y = (lm_array[23, 1] + lm_array[24, 1]) / 2
10    center_z = (lm_array[23, 2] + lm_array[24, 2]) / 2
11
12    lm_array[:, 0] -= center_x
13    lm_array[:, 1] -= center_y
14    lm_array[:, 2] -= center_z
15
16    # Scale to unit box
17    max_dist = np.max(np.abs(lm_array[:, :3]))
18    if max_dist < 1e-6: max_dist = 1
19    lm_array[:, :3] /= max_dist
20
21    return lm_array.flatten()

```

Listing 5: Pose Normalization Function

2. Temporal Smoothing Applied exponential smoothing with $\alpha = 0.7$ to reduce jitter from low-quality video:

$$\text{smoothed}_t = \alpha \cdot \text{current}_t + (1 - \alpha) \cdot \text{smoothed}_{t-1}$$

3. Data Augmentation Added Gaussian noise to training samples to simulate quality degradation:

```
1 def augment_data(features_np):  
2     noise_factor = 0.02  
3     noise = np.random.normal(0, noise_factor, features_np.shape)  
4     return features_np + noise
```

Listing 6: Data Augmentation

4.6.3 Result

The final model maintains robust performance across different video quality levels, achieving consistent detection in production environments.

5 Results and Evaluation

5.1 Performance Metrics

Model Version	Accuracy	Features	Architecture
Run 1: Statistics	78%	264 (mean+std)	Random Forest
Run 2: Sequential	78%	132 × 150	LSTM
Run 3: Velocity	81.5%	264 × 150	Stacked LSTM
Run 4: Extended	76%	273 × 150	Stacked LSTM
Run 5: Normalized	81%+	264 × 150	Stacked LSTM

Table 2: Model Performance Comparison

5.2 Detection Examples

The system has been tested with uploaded videos showing detection confidence scores ranging from 96% to 99% for anomalous content, demonstrating strong discrimination capability.

6 Conclusion

6.1 Key Achievements

- Developed a functional AI-powered violence detection system
- Achieved 81.5% accuracy on benchmark datasets
- Implemented real-time alert notifications via email
- Built a user-friendly web dashboard
- Successfully handled multiple input sources (files, webcam, RTSP streams)

6.2 Lessons Learned

1. Temporal information is essential for action recognition
2. Raw features need semantic transformation (velocity, angles)
3. More features don't always mean better performance
4. Domain adaptation (normalization, augmentation) is critical for deployment
5. Iterative development with careful evaluation leads to better models

6.3 Future Work

- Multi-person tracking and detection
- Additional anomaly types (theft, accidents, etc.)
- Model optimization for edge deployment
- Integration with existing security infrastructure
- Improved alert customization and management

A Training Configuration

```
1 # Model Training Configuration
2 model.compile(
3     optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
4     loss='binary_crossentropy',
5     metrics=['accuracy']
6 )
7
8 # Callbacks
9 early_stopping = EarlyStopping(
10     monitor='val_loss',
11     patience=7,
12     restore_best_weights=True
13 )
14 reduce_lr = ReduceLROnPlateau(
15     monitor='val_loss',
16     factor=0.5,
17     patience=3,
18     min_lr=0.00001
19 )
20
21 # Training
22 history = model.fit(
23     X_train, y_train,
24     validation_data=(X_test, y_test),
25     epochs=60,
26     batch_size=32,
27     callbacks=[early_stopping, reduce_lr]
28 )
```

Listing 7: Final Training Script